Taking away the shortcut

N=24, one change from the 0.9960 record: --shuffle-adj

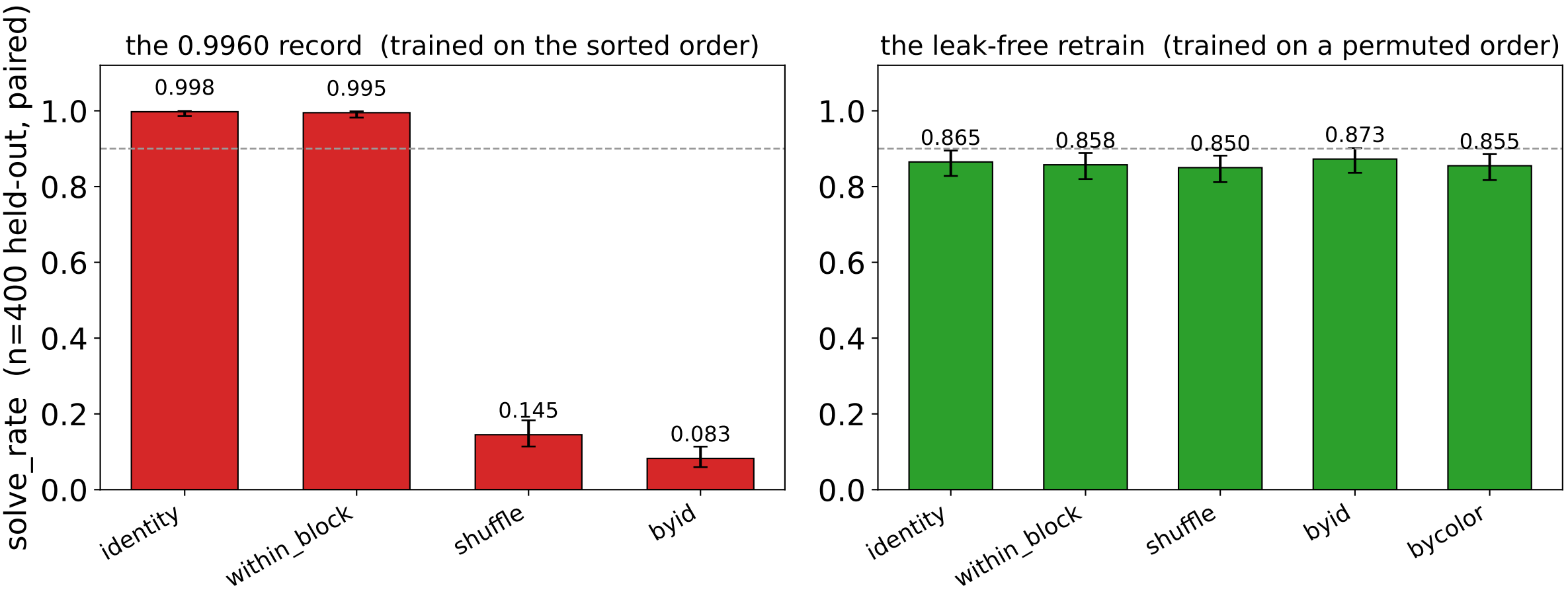
It still solves 85% of held-out instances — and for the first time in this project, solve_rate falls with reasoning depth.

0.8540 [0.8352, 0.8710] on the same n=1500 pool protocol as the record.

Order-insensitive: shuffling, sorting by id, or handing the leak back all tie identity (p = 0.33 / 0.69 / 0.54). At fixed hint density, solve_rate falls 0.946 → 0.827 over T = 2..5 (r = −0.998).

Jobs 49659502 (05:41:35) and 49639313 (03:24:52), both ExitCode 0:0. Package commit b0baa00; training commit 5103e83; seed 0. Held-out pool seed_start=50000000000, n=1500.

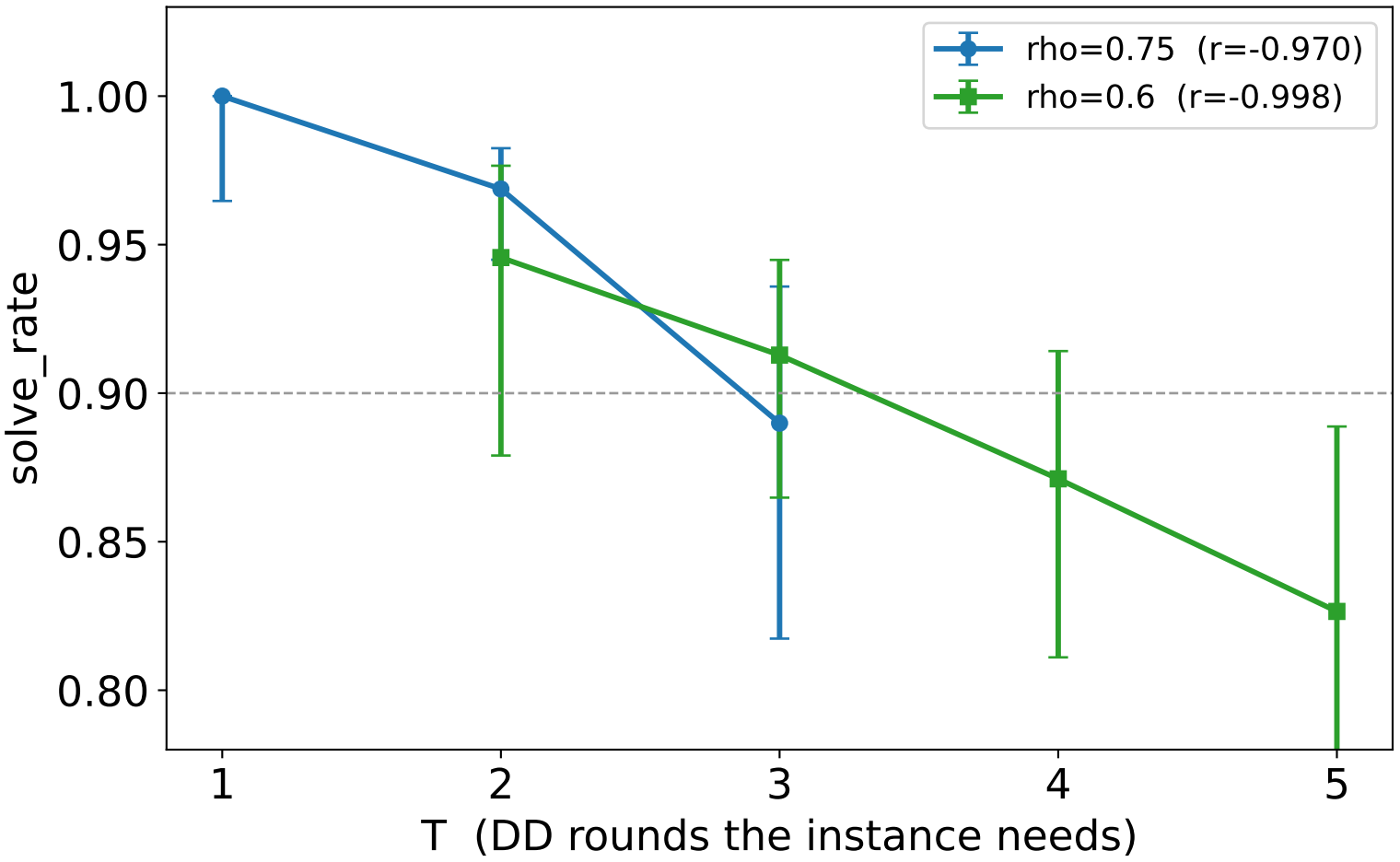
Same graphs, different neighbour order



Every ordering preserves the edge set, the colouring, the hints and the DD trace — the trace is recomputed on the reordered adjacency and asserted equal. `bycolor` re-instates the leak; the leak-free model does not want it (p=0.54).

Left: job 49655447, n=400, bycolor not yet implemented when it ran. Right: job 49659502, n=400, same pool. Paired McNemar exact vs identity.

Where it fails: deeper instances, at identical hint density



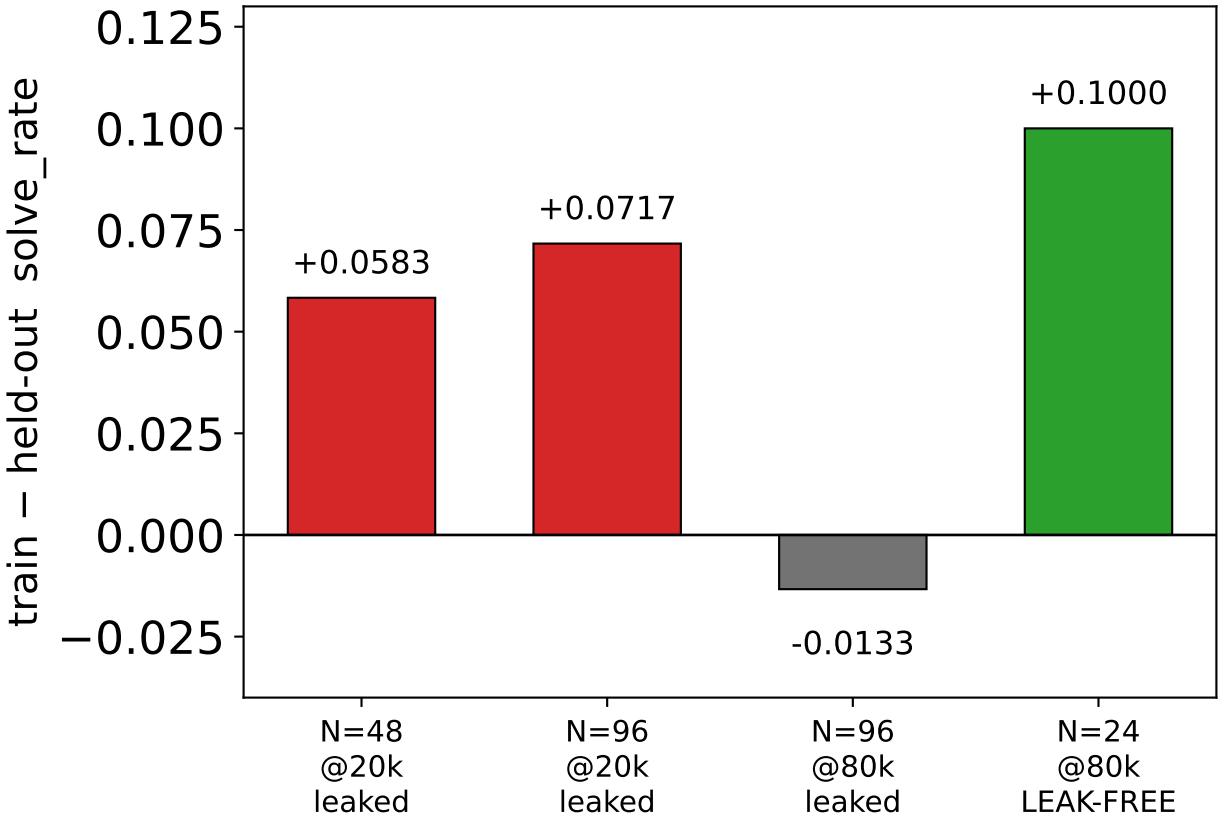
rho	T	n	realized rho	solve
0.75	1	105	0.7500	1.000
0.75	2	352	0.7500	0.969
0.75	3	109	0.7500	0.890
0.6	2	92	0.5842	0.946
0.6	3	195	0.5838	0.913
0.6	4	163	0.5833	0.871
0.6	5	98	0.5833	0.827

At rho=0.75 the realized density is IDENTICAL in all three bins (0.7500), so the fall cannot be a density confound.

Finding 11 measured L*(T) flat in T — but it measured it on a task where depth did not bind. It does here.

Job 49659502, sudoku_depth_eval, --n-pool 600 --min-per-bin 40. Bins too thin to keep: 34 instances at rho=0.75, 52 at rho=0.60. Wilson 95%.

Pre-registration (d), falsified: the honest task is data-hungrier



run (all L=12)	train	held-out	gap	p
N=48 @20k	0.9967	0.9383	+0.0583	1.2e-08
N=96 @20k	0.8967	0.8250	+0.0717	0.00034
N=96 @80k	0.9267	0.9400	-0.0133	0.35
N=24 @80k shuf	0.9567	0.8567	+0.1000	2.6e-09

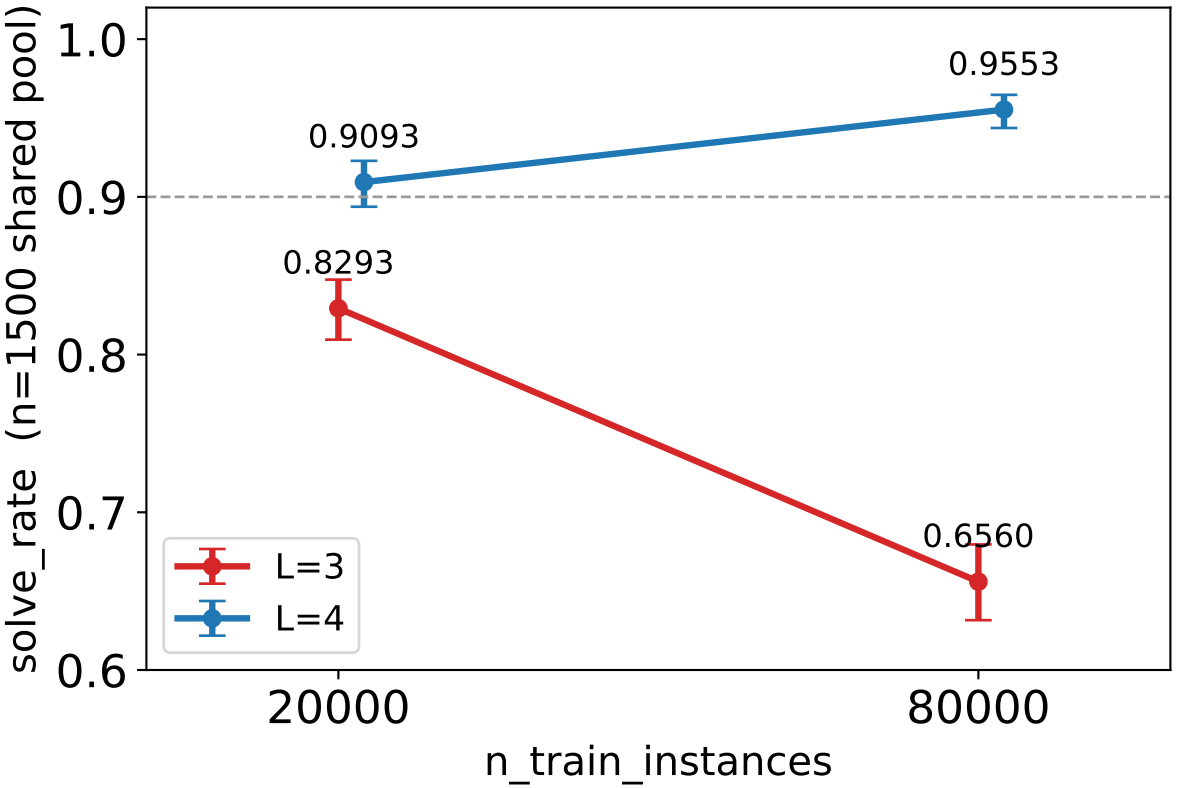
I predicted no gap at 80000, replicating the leaked N=96 result (−0.0133, p=0.355). Wrong by +0.1000 at p=2.6e-09.

Removing the shortcut makes the task harder, so the same 80000 instances no longer cover it — the obverse of "data closes it".

Scored on a recoverable prefix of its own training set: index i is build_instances(cfg, 1, i). Unpaired z-test.

Leaked rows: job 49655447, n=600 per side. Leak-free row: job 49659502, n=600 per side. Grey = not resolvable at p<0.05.

Second run: 4x data at N=48 does not move the knee down



cell	solve	vs L=4 @20k	p
L=4 @20000	0.9093	baseline	—
L=4 @80000	0.9553	+0.0460	5.1e-09
L=3 @20000	0.8293	−0.0800	4.1e-13
L=3 @80000	0.6560	−0.2533	3.0e-85

Pre-registrations (a)-(c) all confirmed: L=3 still misses a rho=0.60 bin so L*(48)=4 stands unconfounded by data budget, L=4 improves, and the 4x gain at N=48 (+0.0460) is smaller than at N=96 (+0.1007).

NOT predicted, and unexplained: 4x data COSTS the under-depth model a quarter of its solve_rate. The pool is held out, so this is not pool memorization. L=3's r(solve_rate, T) is +0.92 at rho=0.75 and +0.83 at rho=0.60 — it does BETTER on deeper instances, which no implementation of DD can do.

Job 49639313, leaked-order run, so these are decoding scores. Paired McNemar exact against L=4 @20000 on the same 1500 instances.

Pre-registrations, as resolved

	prediction (written before the run)	outcome
(a)	below 0.9960 at p<0.05 but above 0.50 — a cost, not a collapse	CONFIRMED 0.8540 [0.8352, 0.8710]
(b)	order-insensitive across every colour-free ordering	CONFIRMED p = 0.66 / 0.33 / 0.69
(c)	re-instating the leak (bycolor) does not help it	CONFIRMED −0.0100, p = 0.54
(d)	no memorization gap at 80000, replicating 49655447	FALSIFIED +0.1000, p = 2.6e-09

What (c) settles, and it is the expensive question:

The model does not reach for the shortcut when it is offered, so the ENCODING is fine: a set-based adjacency representation is not needed. Only the generator had to change, and it already has (5103e83).

What is still owed:

Every solve_rate in STATUS.md except this one is a decoding score, and shuffle_adj is in agg_eval's INSTANCE_KEYS, so no _shuf cell pools with an existing checkpoint — re-measuring a result means retraining it. Depth now binds (page 3), so the leak-free L*(T) series at N=24 is the run finding 11 was trying to be.

My record on mechanism predictions in this project is now 3 for 5. 49702811 is running the SAE on this checkpoint against the leaked record.